week-10-lesson-1

task-1

Trace each step of factorial (4) on paper by listing each function call down to factorial(0). State the value of parameter n and the recursive expression returned at each step. Explain how the pending multiplications resolve during the unwinding phase once the base case factorial(0) returns 1. In C, each function invocation receives its own distinct set of local variables and formal parameters. Explain why the variable result and parameter n in factorial(3) do not overwrite or conflict with result and n in factorial(2). Contrast this behavior with static local variables as discussed in Chapter 7. Describe (experiment) what would happen if the result was declared static inside a recursive function.

task-2

Examine the recursive factorial function:

```
unsigned long int factorial(unsigned int n)
{
     unsigned long int result;
     if (n == 0)
          result = 1;
     else
          {
               result = n * factorial(n - 1);
          }
     return result;
}
```

Identify the base case in this function and explain why every recursive function must have at least one base case. Describe what occurs in computer memory if the base case is omitted or never reached.

Explain why passing an unsigned integer prevents negative inputs, and what would happen if a signed negative integer was passed without validation.

task-3

Write a recursive C function named ‘sum’ that calculates the sum of all positive integers from 1 up to ‘n’ (i.e., 1 + 2 + ... + n).

1. Function Prototype: unsigned int sum(unsigned int n);
2. Base Case: When n == 0, return 0.
3. Recursive Step: Return n + sum(n - 1).
4. Include a main() function that demonstrates calling sum(5) and sum(10) and prints the results.

task-4

Write a recursive C function named ‘power’ that computes x raised to the power of n (x^n) for non-negative integer powers.

1. Function Prototype: long int power(int x, unsigned int n);
2. Base Case: Any number raised to the power of 0 is 1 (return 1 when n == 0).
3. Recursive Step: Return x * power(x, n - 1).
4. Include a main() function that tests power(2, 5) [expected: 32] and power(5, 3) [expected: 125].

task-5

Euclid’s algorithm for finding the Greatest Common Divisor (GCD) of two non-negative integers u and v states:

- gcd(u, v) = u if v == 0
- gcd(u, v) = gcd(v, u % v) if v > 0

1. Function Prototype: unsigned int gcd(unsigned int u, unsigned int v);
2. Implement the recursive function according to Euclids definition.
3. Include a main() function testing gcd(150, 35) [expected: 5] and gcd(1024, 128) [expected: 128].